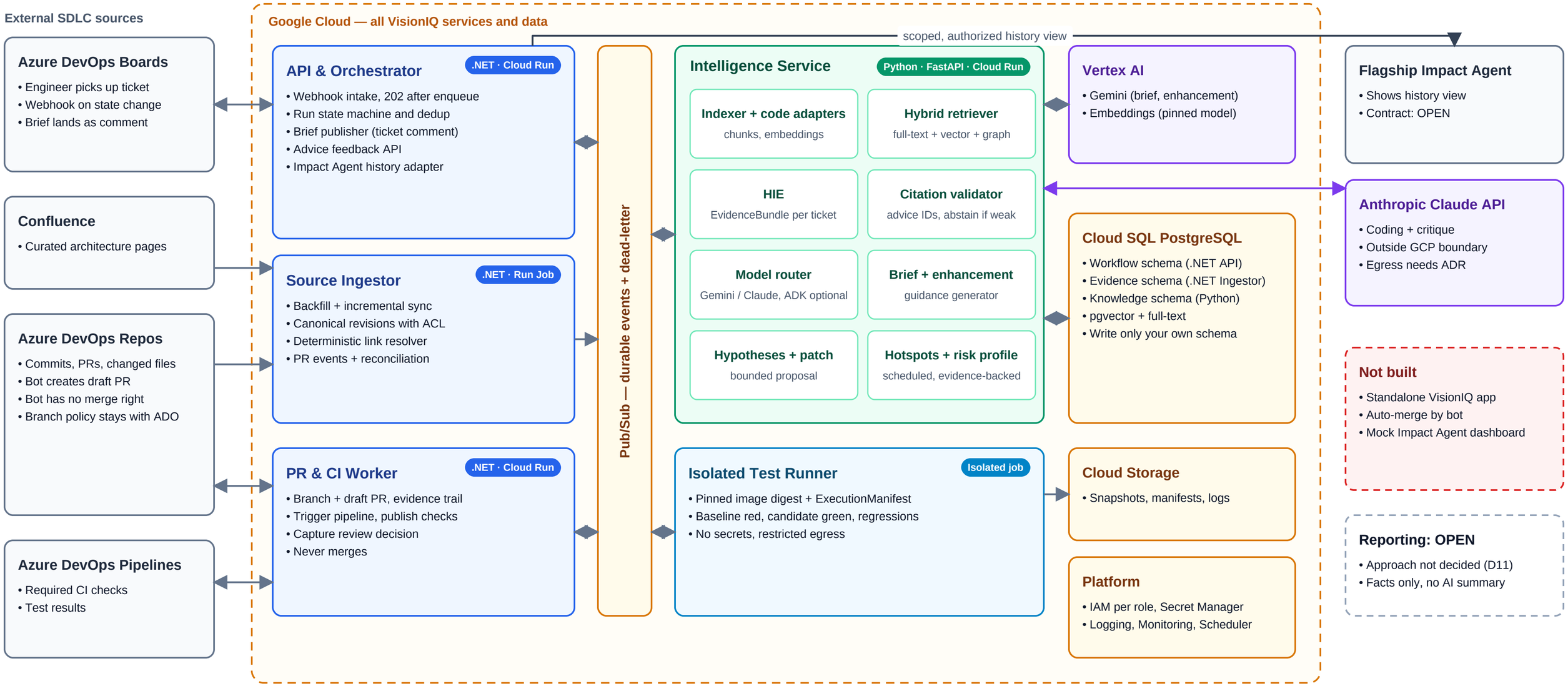
VisionIQ — System Architecture

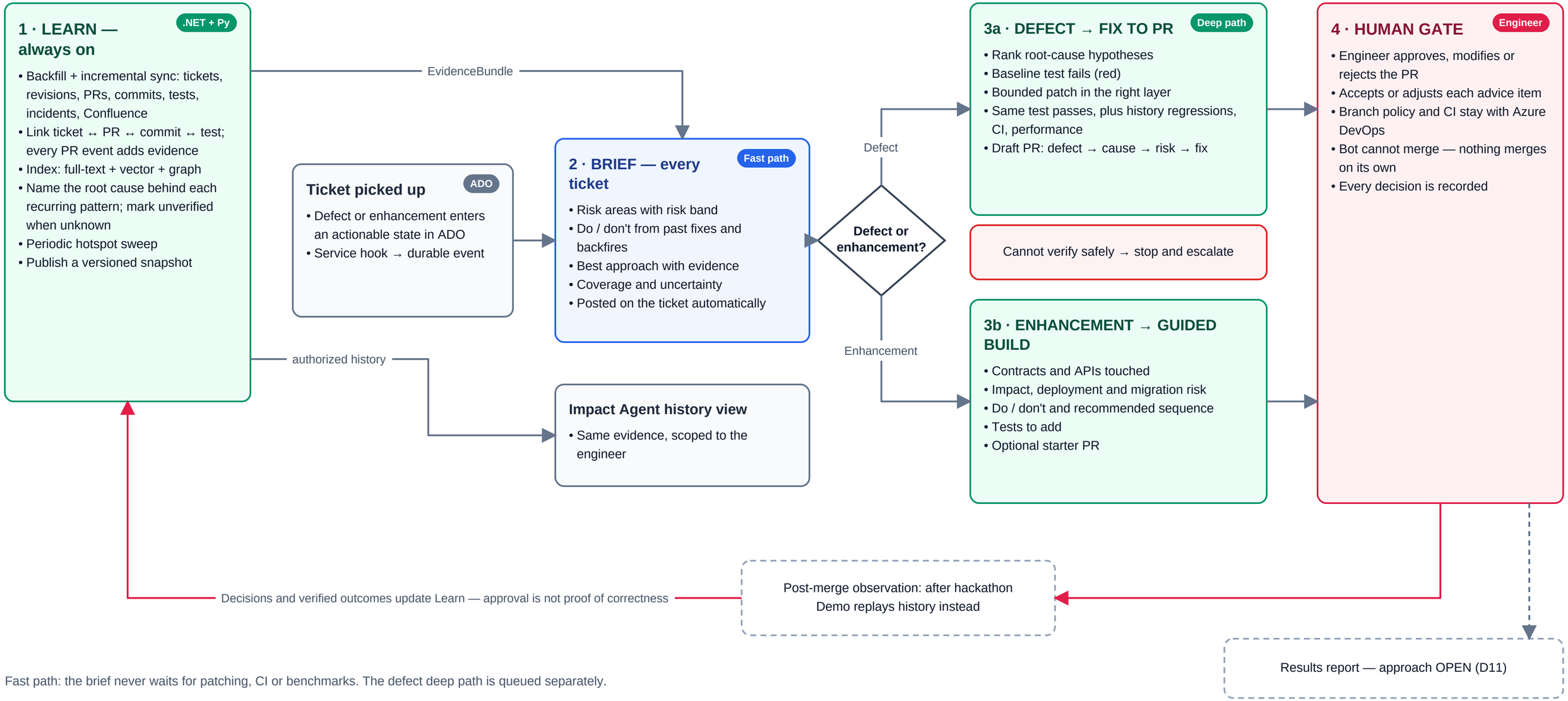
Hybrid build on Google Cloud: .NET for integration and orchestration, Python for intelligence. No separate VisionIQ app.



Aligned to Master Design v1.11 — §5.2, §8.2, §10.8, §10.9

VisionIQ — Four-Phase Workflow

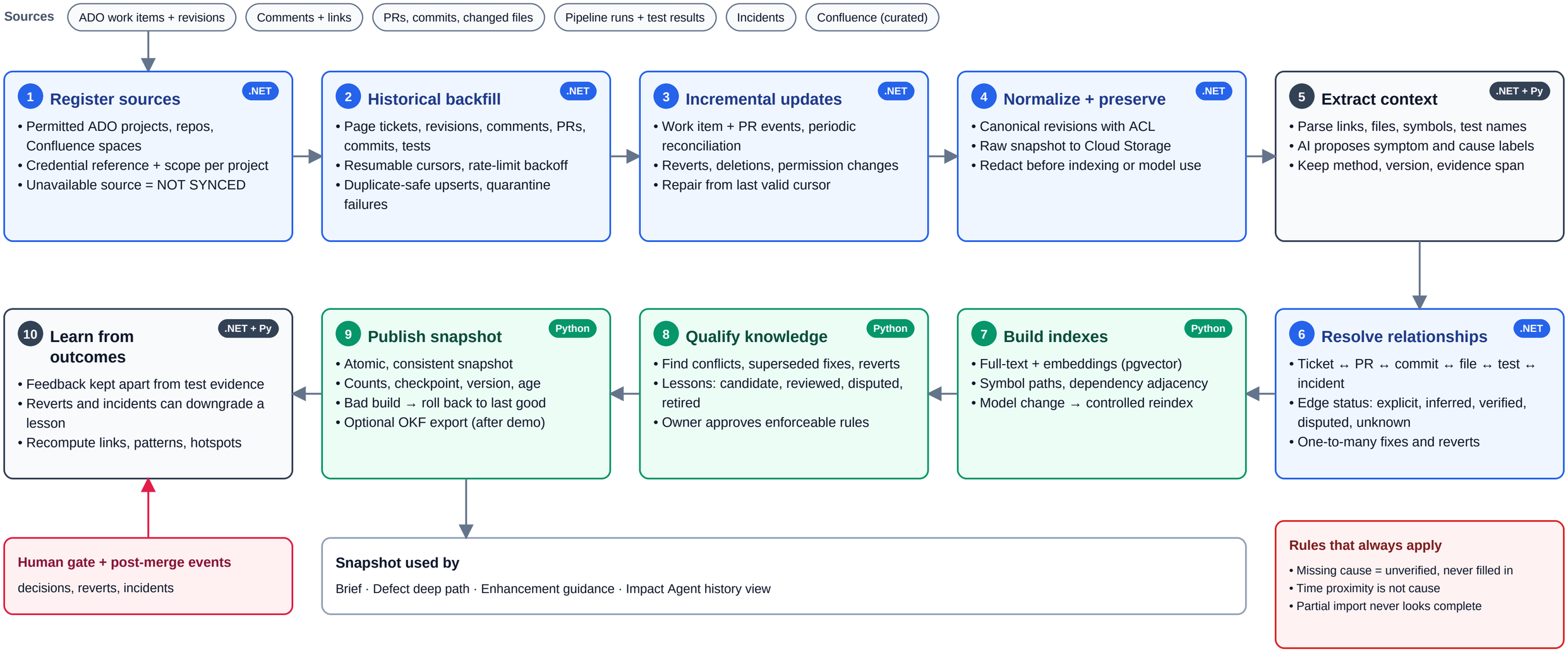
Runs by default when a ticket is picked up: history in, a brief for every ticket, a PR for every defect that can be verified safely, a human at the end.



Fast path: the brief never waits for patching, CI or benchmarks. The defect deep path is queued separately.

LEARN — Knowledge Base Pipeline

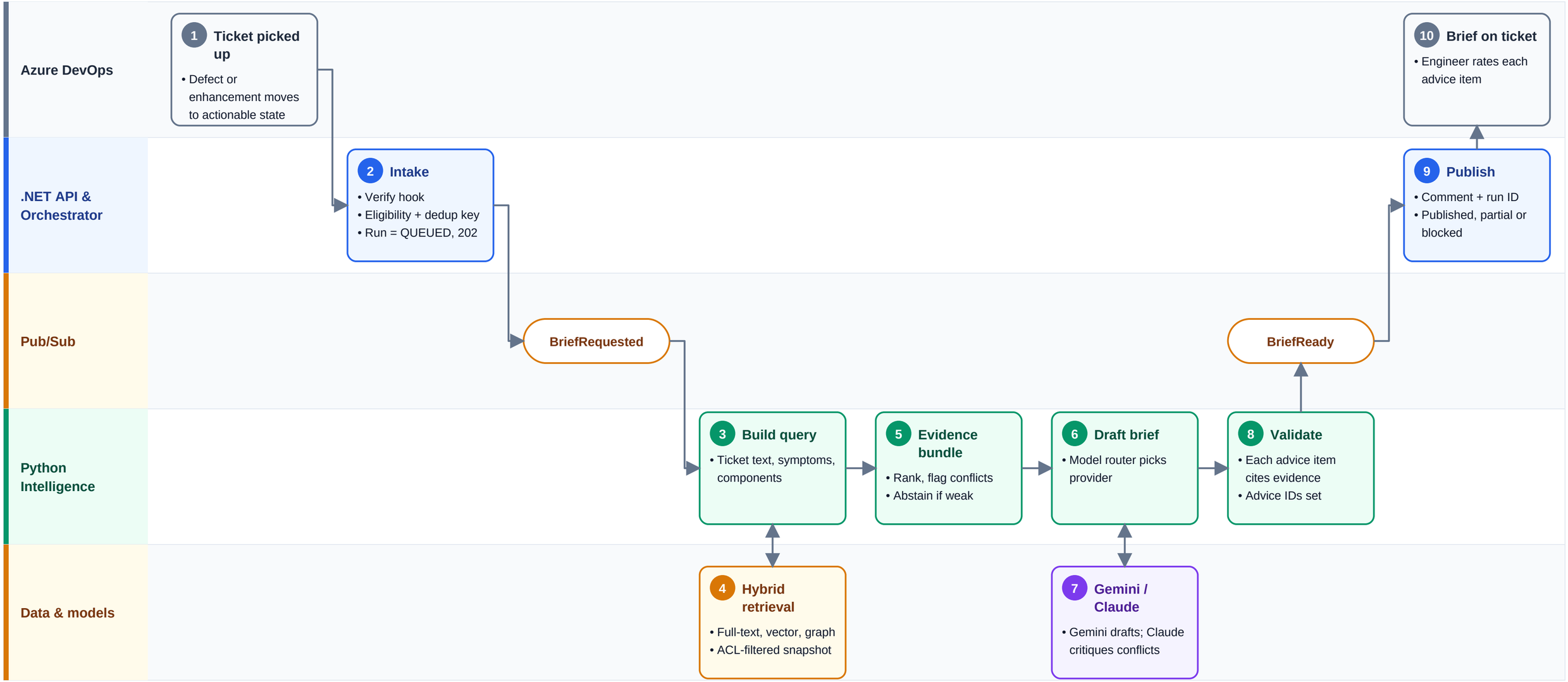
Permission-aware engineering memory. Source systems stay authoritative; VisionIQ keeps attributed, versioned copies.



Aligned to Master Design v1.11 — §3.4, §6.6, §6.12, §6.15

BRIEF — Fast Path on Every Ticket

Automatic on pickup. Runs in workers, never on the webhook thread, and never waits for code, CI or benchmarks.

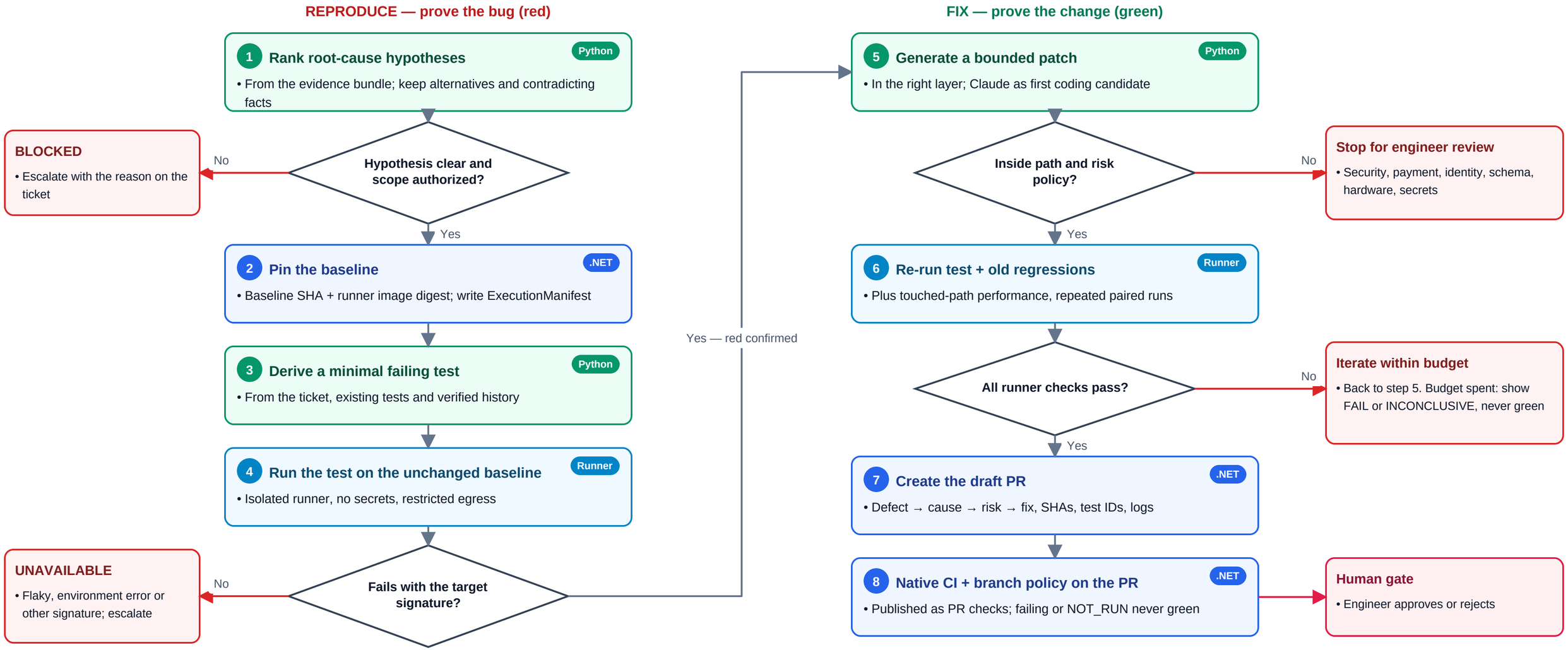


Stops early with PARTIAL or BLOCKED and a visible reason when: scope is unauthorized · sources contradict · prompt injection suspected · model budget exceeded

Aligned to Master Design v1.11 — §3.5, §7.1, §7.4, §8.1, §10.9

DEFECT — Fix to PR with Red-Green Proof

Queued after the brief. Every stop is visible on the ticket; nothing is reported green that did not run green.

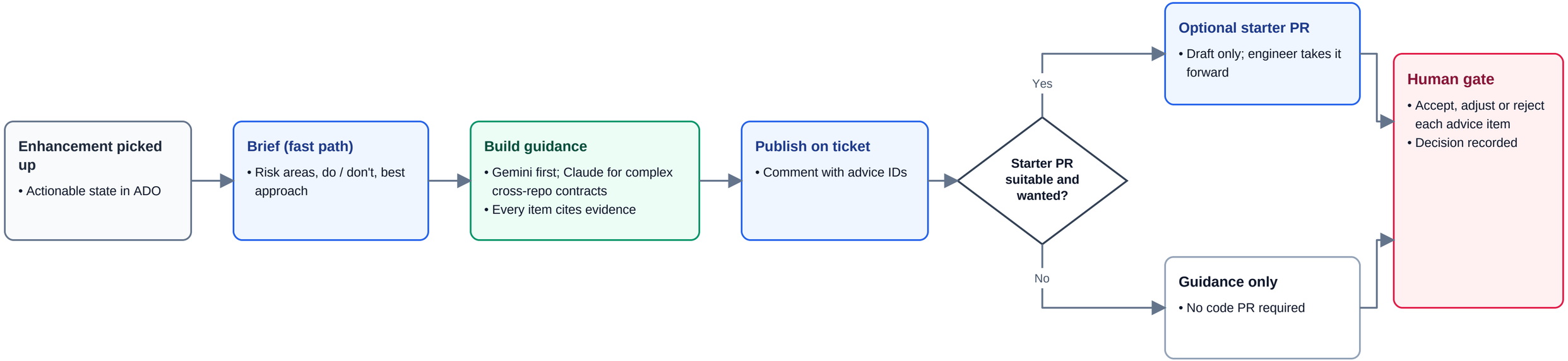


Test outcomes: PASS · FAIL_TARGET_SIGNATURE · FAIL_DIFFERENT_SIGNATURE · FLAKY · ENVIRONMENT_ERROR · TIMEOUT · NOT_RUN · UNAVAILABLE · INCONCLUSIVE

Aligned to Master Design v1.11 — §3.3, §8.6.2, §8.6.3, §8.6.4, §8.7

ENHANCEMENT — Guided Build

Guidance lands on the ticket. A starter PR is optional, never required. The engineer decides on every advice item.



What the guidance must contain

1 Acceptance-criteria gaps
What the ticket does not yet specify

2 Contracts and APIs touched
Interfaces, payloads and consumers affected

3 Deployment and migration risk
Schema, configuration and rollout order

4 Related past changes
Earlier work in this area and how it ended

5 Do / don't
From past successes, regressions and backfires

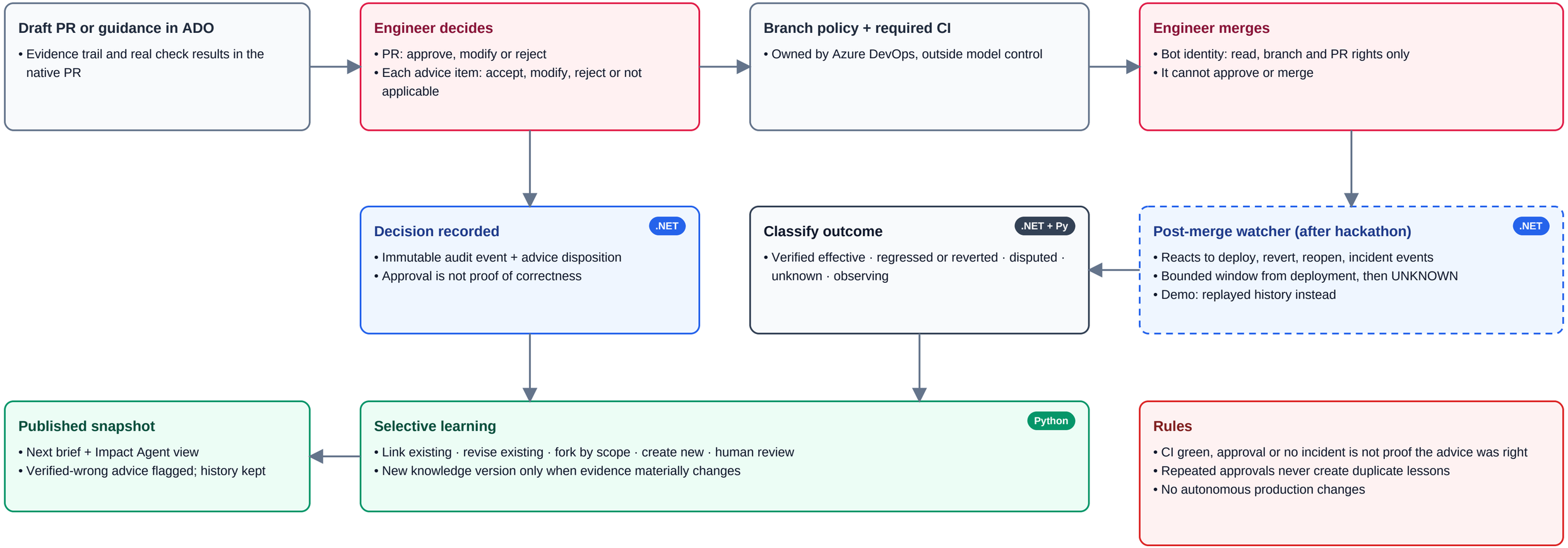
6 Recommended sequence
Order of work that limits risk

7 Tests to add
Contract, regression and touched-path checks

8 Reviewer decisions
Who must decide what before merge

HUMAN GATE — Decisions and Outcome Learning

AI proposes, engineers decide. Every decision is recorded; only new verified or disputed evidence changes knowledge.



Two independent states per advice item

Engineer disposition — what the engineer did

PRESENTED · ACCEPTED · MODIFIED · REJECTED · NOT_APPLICABLE

Evidence judgment — what tests, review and later outcomes show

UNVERIFIED · SUPPORTED_BY_TEST · CONTRADICTED_BY_TEST · VERIFIED_CORRECT · VERIFIED_WRONG · DISPUTED · SUPERSEDED

Work by Delivery Gate and Stack

Gates G0–G5 by part of the system. Each gate ends with something demonstrable, not a percentage.

	G0 Access + fixtures Inventory and runnable fixture confirmed	G1 Trustworthy history Idempotent replay, ACL test, 5 links reviewed	G2 Learn + auto brief Both tickets get real-history briefs	G3 Defect PR + gate Green tests, draft PR, bot cannot merge	G4 Failure memory Old regression catches unsafe patch	G5 Measure + Impact Agent Measured results; flagship view if access given
Python Intelligence service + GCP / DevOps	• GCP project, IAM, secrets, Terraform • Build pipeline for both stacks • Network egress, schema isolation	• Embeddings + full-text + pgvector • Snapshot publish + rollback • Lesson lifecycle, adapter checks	• Retrieval + evidence bundle • Model router + citations • Brief, guidance, risk profile	• Isolated runner + manifest • Patch proposal • Flaky and signature policy	• Unsafe vs safe candidate run	• Recall, grounding, p95 latency • Dashboards + alerts
.NET Source Ingestor — knowledge foundation	• ADO inventory + fix/revert pair • Cloud SQL + pgvector • Three-schema migrations	• Backfill + incremental sync • Canonical revisions + ACL • Link resolver, outage recovery	• PR-driven evidence sync • Lexical index	• Historical test discovery from KB links	• Recover old failure + its test	• Point-in-time replay on benchmark
.NET API + PR & CI Worker — integration	• Bot without merge right • Pub/Sub + Cloud Run deploy • Event contracts, Impact Agent contact	• Run ledger + event schema • Idempotency keys • Reconciliation job	• Webhook intake + dedup • Brief publisher on ticket • Advice feedback, missed-event test	• Red-green orchestration • Draft PR + checks • Review decision capture	• Show regression result in PR checks	• Impact Agent adapter • Telemetry + brief latency
Team Whole team	• Pick live defect + live enhancement • Contact Impact Agent owner • Record ADRs	• Review 5 known links • Label a small benchmark • KB readiness tests	• Trigger both tickets • No VisionIQ page visited	• Engineer approves • Bot merge attempt fails	• Or state clearly that none exists	• Security tests, demo run • Unmet dependencies stated • Decide + build reporting (D11)

After the hackathon (G6): live post-merge observation over real time. Learning is shown in the demo by replaying history (S12-16, S17-09).

PR-Driven Learning — When the Knowledge Base Changes

Every PR event adds evidence immediately. Reusable lessons change only when new verified or disputed evidence exists.

	PR created	PR updated / push	CI / test result	PR abandoned / rejected	PR merged	Revert, reopen, incident
Immediate evidence always, idempotent	• PR ID, source and target SHAs • Work item links, changed files • Patch intent	• New iteration and diff • Comments and votes • Old diff marked superseded	• Runner manifest, test IDs • PASS, FAIL or INCONCLUSIVE • Report links	• Review reasons kept • Hypotheses stay unverified	• Final SHA, policy, CI receipt	• Event-driven, no polling • Window starts at deployment; ends UNKNOWN if no evidence • Cause stays uncertain until shown
Effect on lessons deferred, gated by proof	• Link to past patterns • Start scoped search	• Reindex changed files • Refresh test selection	• Supports or refutes specific claims	• Candidate unsafe approach • Never auto-labelled a backfire	• Reassess related lessons • Approval is not correctness	• Dispute, supersede or confirm a lesson • New version, history kept
Proves the fix correct?	No	No	Only what the tests cover	No	No	Only with verified outcome

Live: after hackathon · Demo: replayed history

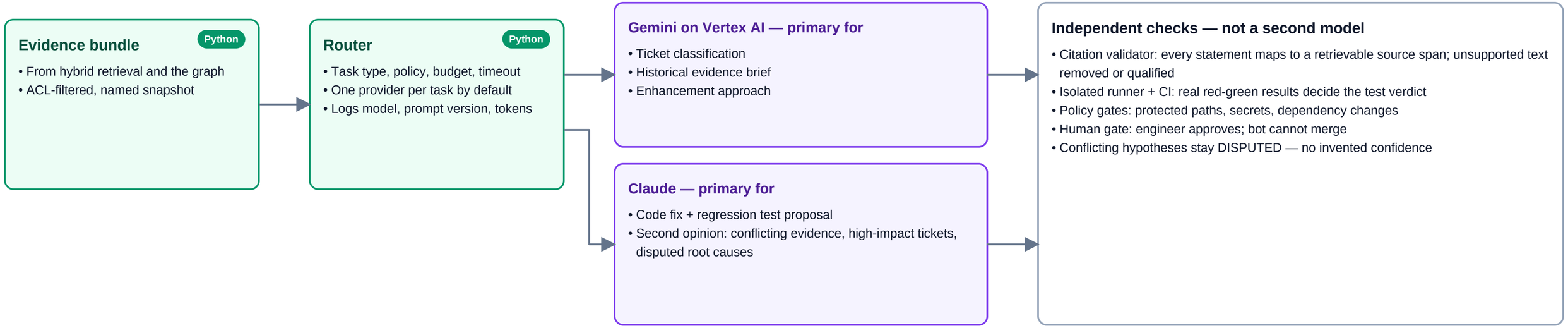
Evidence sync (.NET) Lesson reassessment (Python)

Rules

- No global retraining on each PR: incremental event ingestion, selective reindex of affected files and chunks, verified lesson consolidation.
- Replaying the same event twice changes nothing. Repeated tickets link to one pattern instead of creating duplicate lessons.
- OKF Markdown/YAML is an optional curated export of approved lessons — not the database, the graph or the search engine.

Model Routing — Gemini, Claude and Optional ADK

Models propose; code decides what exists, who may see it, whether tests passed and whether a PR merged.



Provider outage or budget exceeded

- Try the other provider only if policy and budget allow
- Otherwise: PARTIAL status with deterministic evidence — never invented advice

Claude data boundary

- Direct Claude API calls leave Google Cloud
- Source and code may be sent only after the egress decision (D04) is approved

Google ADK — optional, inside the Python service, after the win condition works

- May run bounded retrieval or investigation steps behind the same ModelProvider interface
- Build deterministic orchestration first; keep ADK only if it is simpler or measurably better (story EXT-03)
- Never used for ingestion, ACL filtering, source of truth, CI verdicts, merge approval or publishing verified lessons